

Training the tooth-root model (design A)

Goal: a small nnU-Net that outputs the CEJ-based **root** given a CBCT plus the tooth and bone masks from DentalSegmentator. This replaces the geometric heuristic in `roots.py` with a learned cutoff.

Input channels: `0=CBCT`, `1=tooth mask`, `2=bone mask`. Label: `1=root`. Because the root is a subset of the (provided) tooth, the task is well-posed: the model only has to decide which part of a known tooth is root.

0. Environment

```
pip install nnunetv2
# nnU-Net needs three folders; set them once (Windows: use setx or the GUI):
export nnUNet_raw=/path/nnUNet_raw
export nnUNet_preprocessed=/path/nnUNet_preprocessed
export nnUNet_results=/path/nnUNet_results
```

Windows (cmd / Anaconda Prompt)

`export` is Linux/Mac only. On Windows use the provided helper — from your working terminal (not by double-clicking):

```
conda activate your_env
cd C:\Users\seoul\Desktop\Joontae\cbct-trabecular-pipeline\cbct-trabecular-
pipeline
training\set_env.bat
```

`set_env.bat` sets `nnUNet_raw`, `nnUNet_preprocessed`, `nnUNet_results` for the current terminal and creates the folders (under a `nnUNet` subfolder by default; edit `BASE` inside the script to change the location). Re-run it in every new terminal. To make them permanent instead, use `setx` once (then open a new terminal so they take effect):

```
setx nnUNet_raw "C:\Users\seoul\Desktop\Joontae\nnUNet\nnUNet_raw"
setx nnUNet_preprocessed
"C:\Users\seoul\Desktop\Joontae\nnUNet\nnUNet_preprocessed"
setx nnUNet_results "C:\Users\seoul\Desktop\Joontae\nnUNet\nnUNet_results"
```

Windows versions of the main commands (use `%var%`, backslashes, `^` not `\` for line continuation — or just keep each command on one line):

```
python training\prepare_dataset.py --images data\images --roots data\trabecular ^
--dentalseg-config config.dentalsegmentator.yaml ^
--out "%nnUNet_raw%" --dataset-id 201 --dataset-name ToothRoot --mask-channel-
```

```
noNorm
```

```
nnUNetv2_plan_and_preprocess -d 201 --verify_dataset_integrity
nnUNetv2_train 201 3d_fullres 0
nnUNetv2_predict -i IN_DIR -o OUT_DIR -d 201 -c 3d_fullres -f all
```

1. Build the dataset

You have root labels already. Point the builder at your images + root labels; it produces the tooth/bone channels from DentalSegmentator (or from precomputed masks). Match is by the leading patient id in each filename.

Easiest — one folder (your pipeline outputs). If a single folder holds, per patient, `<id>.nii.gz` (CBCT), `<id>_teeth.nii.gz`, `<id>_Mandible.nii.gz` or `<id>_Maxilla & Upper Skull.nii.gz`, and `<id>_trabecular.nii.gz`, just point `--dir` at it. The builder uses the existing teeth/bone as channels (no need to re-run DentalSegmentator) and `_trabecular` as the label:

```
python training\prepare_dataset.py --dir training\data --out "%nnUNet_raw%" --
dataset-id 201 --dataset-name ToothRoot --mask-channel-noNorm
```

The suffixes are configurable (`--image-suffix`, `--root-suffix`, `--tooth-suffix`, `--bone-suffix`) if your naming differs; the bone suffix accepts a comma list and picks whichever exists per case.

Or separate folders (images in one, labels in another):

```
python training/prepare_dataset.py \
  --images data/images \
  --roots data/root_labels \
  --dentalseg-config config.dentalsegmentator.yaml \
  --out "$nnUNet_raw" --dataset-id 201 --dataset-name ToothRoot \
  --mask-channel-noNorm
```

If you already have tooth/bone masks on disk, pass `--tooth DIR --bone DIR` instead of `--dentalseg-config` (faster; no inference during dataset build). For a CBCT-only model (no DentalSegmentator channels), add `--channels image`.

2. Plan & preprocess

```
nnUNetv2_plan_and_preprocess -d 201 --verify_dataset_integrity
```

Optional stronger network (recommended if you have a big GPU): plan a Residual Encoder preset, then pass its plans to train/predict:

```
nnUNetv2_plan_experiment -d 201 -pl nnUNetPlannerResEncL
# then add -p nnUNetResEncUNetLPlans to the train/predict commands below
```

3. Train

With 100+ cases, train from scratch (no pretrained weights needed):

```
# 5-fold cross-validation (each ~a day on a good GPU); run folds in parallel if
you can
nnUNetv2_train 201 3d_fullres 0
nnUNetv2_train 201 3d_fullres 1
nnUNetv2_train 201 3d_fullres 2
nnUNetv2_train 201 3d_fullres 3
nnUNetv2_train 201 3d_fullres 4
```

`nnUNetv2_find_best_configuration 201` then reports the best setup. (If you ever train with far fewer cases, warm-start instead: align plans with the pretraining dataset, then `nnUNetv2_train ... -pretrained_weights CHECKPOINT` — see nnU-Net's [pretraining_and_finetuning.md](#).)

4. Inference

The model expects the **same 3 channels** at inference, so you must build the tooth/bone channels for each new scan (run DentalSegmentator, stack). nnU-Net reads them as `CASE_0000.nii.gz` (CBCT), `CASE_0001.nii.gz` (tooth), `CASE_0002.nii.gz` (bone) in the input folder:

```
nnUNetv2_predict -i IN_DIR -o OUT_DIR -d 201 -c 3d_fullres -f all
```

5. Integrate into the pipeline

Once trained, the root model drops in as a new root "backend": the pipeline runs DentalSegmentator (as now), stacks CBCT+tooth+bone, runs the root model, and uses its output in place of `extract_root`. The dominant-arch selection, naming, and outputs stay identical. Ask and this wiring can be added as `root.method: model` with a `root.model_dir` pointing at your trained `Dataset201_ToothRoot/...` folder.

Tips

- Consistency of the CEJ cut across your 100+ labels matters more than anything.
- Keep image, tooth, bone, and root all on the **same voxel grid/affine** per case (the builder verifies this and skips mismatches).
- Hold out a few cases the model never sees to sanity-check Dice and, more importantly, to eyeball the cut in 3D.